

Ch 06: File Input / Output Processing

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

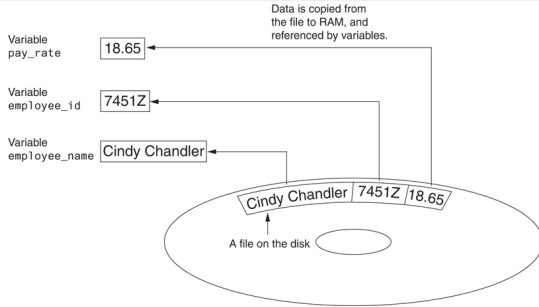


EAST TEXAS A&M

Introduction to File Input and Output

File Input / Output

When a program needs to save data for later use, it writes the data in a file. The data can be read from the file at a later time.



- For program to retain data between the times it is run, you must save the data
 - Data is saved to a file, typically on computer disk
 - Saved data can be retrieved and used at a later time
- “Writing data to”: saving data on a file
 - **Output file**: a file that data is written to
- “Reading data from”: process of retrieving data from a file
 - **Input file**: a file from which data is read
- Three steps when a program uses a file
 - 1 Open the file
 - 2 Process the file
 - 3 Close the file

Figure 1: Reading data from a file (Gaddis, 2021, pg. 226)



Types of Files and File Access Methods

Types of Files

- In general, there are two types of files that can be created
 - 1 **Text file:** contains data that has been encoded as (ASCII/Unicode) text
 - 2 **Binary file:** contains data that is not text encoded, cannot open in a text editor

File Access Methods

- Two ways to access data stored in a file
 - 1 **Sequential access:** file read sequentially from beginning to end, can't skip ahead
 - 2 **Direct access:** (also random access) can jump directly to any piece of data in the file



Filenames and File Objects

- **Filename extensions:** short sequences of characters that appear at the end of a filename preceded by a period
 - Extension indicates type of data stored in the file
 - OS may use extension to open associated program to view, but you can create file with any extension you like
- **File object:** object associated with a specific file
 - Provides a way for a program to work with the file
 - File object referenced by a variable

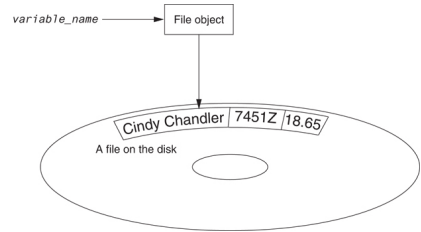


Figure 2: A variable name references a file object that is associated with a file (Gaddis, 2021, pg.326)



Opening a File

- **open() function:** used to open a file
 - Creates a file object and associates it with a file on the disk
- **Mode:** string specifying how the file will be opened

General format:

```
file_variable = open(filename, mode)
```

Table 1: Some of the Python file modes

ModeDescription	
'r'	Open a file for reading only. The file cannot be changed or written to.
'w'	Open a file for writing. If the file already exists, erase its contents. If it does not exist, create it.
'q'	Open a file to be written to. All data written to the file will be appended to its end. If the file does not exist, create it.



EAST TEXAS A&M

Specifying the Location of a File

- If `open()` receives a filename that does not contain a path, assume that file is in the same directory as program
- If program is running and file is created, it is created in the same directory as the program
 - Can specify alternative path and file name in the open function argument
 - Prefix the path string literal with the letter `r` for raw strings, especially needed on Windows computers that use `\` as path separators

Note: It is generally **BAD** to encode an absolute path name in a program. This will often make the program not portable to any other location unless the path name is changed again before you run the program.



Writing Data to a File

- **write() method:** a function that belongs to an object (returned by `open()`)
 - Performs operations using that object
- File object's `write()` method used to write data to the file
- File should be closed using file object `close()` method

Format:

```
test_file = open('test.txt', 'w')
test_file.write('Test String')
test_file.close()
```



Reading Data From a File

- **read() method:** file object method that reads entire file contents into memory
 - Only works if files has been opened for reading
 - Contents returned as a string
- **readline() method:** file object method that reads a single line from the file
 - Line returned as a string, including trailing '\n'
- **Read position:** marks the location of the next item to be read from a file



Note on Opening and Closing Files

Since our textbook was written, the recommended way to open and close files for access has changed. You can and should use `with open()` as file:

```
with open('test.txt', 'r') as test_file:  
    content = test_file.read()  
    print(content)
```

- No `close()` is issued, the `with` block automatically closes files opened by the `with` statement.
- Consider the modern recommended way to open files [Python open\(\) Function Explained](#) (Ocean, 2025)
- Prevents certain bugs, when an error occurs file is guaranteed to be closed
- You should use `with` blocks in your assignments for this class



Concatenating a Newline to and Stripping it From a String

- In most cases, data items written to a file are values reference by variables
 - Usually necessary to concatenate a '`\n`' to data before writing it
 - Carried out using the `+` operator in the argument of the `write()` method
- In many cases need to remove '`\n`' from string after it is read from a file
 - `rstrip()` method: string method that strips specific characters from end of string



Appending Data to an Existing File

- When open with 'w' mode, if the file already exists it is overwritten
- To append data to a file use the 'a' mode
 - If file exists, it is not erased, and if it does not exist it is created
 - Data is written to the file at the end of the current contents



Writing and Reading Numeric Data

- Numbers must be converted to strings before they are written to a file
- **str() function:** converts value to a string
- Numbers are read from a text file as strings
 - Must be converted to numeric type in order to perform mathematical operations
 - Use `int()` and `float()` functions to convert string to numeric value



Using Loops to Process Files

- Files typically used to hold large amounts of data
 - Loop typically involved in reading from and writing to a file
- Often the number of items stored in file is unknown
 - The `readline()` method uses an empty string as a **sentinel** when end of file is reached
 - Can write a while loop with the condition

```
while line != '':
```

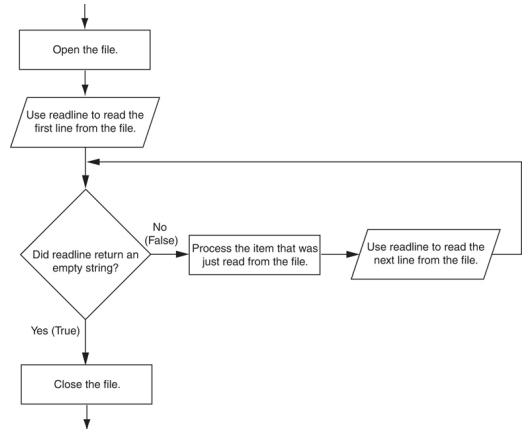


Figure 3: General logic for detecting the end of a file reading line-by-line (Gaddis, 2021, pg.345)



Using Python's for Loop to Read Lines

- Better method for line oriented processing
- Python allows programmer to write a for loop that automatically reads lines in a file and stops when end of file is reached

```
for line in file_object:  
    statements
```

- The loop iterates once over each line in the file and stops when end of file reached



Summary

- For programs to retain data, must be saved in permanent (non-volatile) memory, e.g. on computer disk as a file
- Three steps
 - ① Open the file using `open()`
 - ② process the file (read or write usually one, sometimes both)
 - ③ Close the file using `close()`
 - Should use `with open()` as `file:` blocks in new Python code
- Files can be open as text, or as binary
- Files can be accessed sequentially, or using direct access (random access)
- Use Python `for` loop on file object to read and process line-oriented input



References

Gaddis, T. (2021). *Starting out with python* (5th). Pearson.

Ocean, D. (2025). *Python open() function explained: How to open, read and write files.*

Retrieved June 25, 2025, from <https://www.digitalocean.com/community/tutorials/python-read-file-open-write-delete-copy>



EAST TEXAS A&M